OpenShift / Kubernetes Hands-On Training

5-Day Interactive Workshop

Duration: 1 hour/day × 5 days
Format: Interactive, hands-on
Prerequisites: Access to an OpenShift cluster, `oc` CLI installed
By: Siddharth Patel
CKAD | CKA | CKS | MANA Delivery Champion - Kubernetes 2022

Table of Contents

Day 1: Foundations & First Deployment

What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications.

The Problem Kubernetes Solves

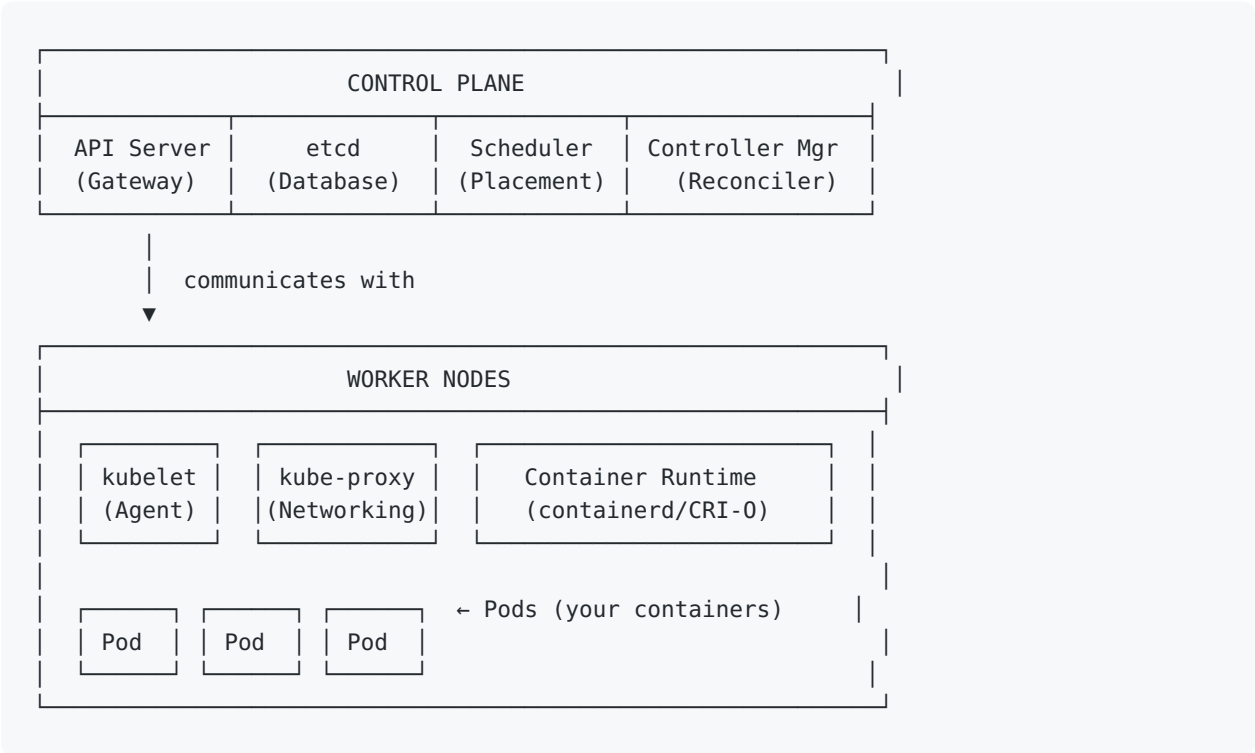
Without Kubernetes:

- Manual container placement across servers
- No automatic recovery when containers crash
- Manual scaling during traffic spikes
- Complex networking between containers
- Inconsistent deployments across environments

With Kubernetes:

- **Self-healing:** Automatically restarts failed containers
- **Auto-scaling:** Scales apps based on demand
- **Service discovery:** Built-in DNS and load balancing
- **Rolling updates:** Deploy new versions with zero downtime
- **Declarative configuration:** Define desired state, K8s makes it happen

Kubernetes Architecture



Control Plane Components:

Component	Role
API Server	Front door to the cluster. All `kubectl` / `oc` commands go through it
etcd	Key-value store holding all cluster state (the “database”)
Scheduler	Decides which node a new pod should run on based on resources
Controller Manager	Runs control loops that reconcile desired state vs actual state

Worker Node Components:

Component	Role
kubelet	Agent on each node; ensures containers are running as expected
kube-proxy	Manages network rules so pods can communicate
Container Runtime	Actually runs containers (CRI-O in OpenShift)

POD

A pod is a single instance of an application, and they are the smallest deployable units of computing that you can create and manage in Kubernetes. You will deploy containers inside these pods where you will deploy your application.

- The key thing about pods is that when a pod does contain multiple containers, all of them are always run on a single worker node and it never spans multiple worker nodes.
- We’re not implying that a pod always includes more than one container, it’s common for pods to contain only a single container.

Understanding Multi-Container Pods

Normally we use single container with a pod as they are easier to build and maintain although there are some cases where you might want to run multiple containers in a Pod.

Type	Description
Init containers	An init container is an additional container in a Pod that completes a task before the “regular” container is started. The regular container will only be started once the init container has been started. This is a great way to initialize a Kubernetes Pod. You can pull any files (keystores, policies, and so forth), configurations, and so on with an init container.
Sidecar container	A container that enhances the primary application, for instance for logging purposes.
Ambassador container	A container that represents the primary container to the outside world, such as a proxy.
Adapter container	Used to adopt the traffic or data pattern to match the traffic or data pattern in other applications in the cluster.

When using multi-container Pods, the containers typically share data through shared storage.

> **Note:** Containers within the same Pod also share the same network namespace. This means they can communicate with each other via `localhost` and share the same IP address and port space.

What is OpenShift?

OpenShift is Red Hat’s enterprise Kubernetes platform. It takes vanilla Kubernetes and adds:

Feature	Kubernetes	OpenShift
Web Console	Basic Dashboard	Rich Developer + Admin Console
Networking	Ingress	Routes (simpler, built-in TLS)
CI/CD	None built-in	BuildConfigs, Pipelines (Tekton)
Security	Manual RBAC setup	SCCs, built-in OAuth, RBAC out of box
Image Management	Manual registry	

Feature	Kubernetes	OpenShift
		Image Streams, integrated registry
Developer Experience	CLI-focused	Web Console + CLI + IDE plugins
Updates	Manual	Over-the-air cluster upgrades

Why OpenShift on top of Kubernetes?

1. **Enterprise Security** — Security Context Constraints (SCCs) prevent containers from running as root by default
2. **Built-in Routes** — No need for external ingress controllers; expose apps with one command
3. **Developer Productivity** — Web console with topology view, built-in pipelines, source-to-image (S2I)
4. **Operator Framework** — Manage complex applications with Operators from OperatorHub
5. **Support & Stability** — Red Hat tested, certified, and supported

---

OpenShift Web Console Tour

Administrator Perspective

- **Home** → Dashboards, cluster status, events
- **Workloads** → Deployments, Pods, ReplicaSets, Jobs
- **Networking** → Services, Routes, Ingress
- **Storage** → PersistentVolumeClaims
- **Monitoring** → Metrics, Alerts, Dashboards (Prometheus/Grafana)

Developer Perspective

- **Topology** → Visual map of your application components
- **+Add** → Deploy from Git, Container Image, Catalog, YAML
- **Builds** → BuildConfigs, build history
- **Project** → Resource usage, quotas

Creating Using YAML File

To create a Kubernetes resources using a YAML file we would need the KIND and apiVersion. To get the KIND value of a resources we will list down the api-resources:

```
oc api-resources | grep -iE 'deployment|KIND'
```

To get detailed information about a resource including its apiVersion and spec fields:

```
oc explain deployment
```

Hands-On: First Deployment

Step 1: Login and Create a Project

```
oc login <cluster-url>
oc new-project my-first-app   # Admin only
```

Step 2: Deploy Nginx

nginx-deployment.yaml:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: bitnami/nginx:latest
```

```
          ports:
          - containerPort: 8080
```

```
oc apply -f nginx-deployment.yaml
oc get pods -w
```

Step 3: Expose with a Service

nginx-service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
  type: NodePort
```

```
oc apply -f nginx-service.yaml
```

Step 4: Create a Route

nginx-route.yaml:

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: nginx-service
spec:
  to:
    kind: Service
    name: nginx-service
  port:
    targetPort: 8080
```

```
oc apply -f nginx-route.yaml
curl $(oc get route nginx-service -o jsonpath='{.spec.host}')
```

Essential Commands

```
oc get pods                    # List pods
oc get svc                     # List services
oc get routes                  # List routes
oc describe pod <pod-name>     # Detailed pod info
oc logs <pod-name>             # View container logs
oc delete -f <file.yaml>       # Clean up resources
```

Entering a Pod/Container Shell

To exec into a pod's container:

```
kubectl exec nginx-lab-1-58f9bf94f7-jk85s -- /bin/bash
```

For a pod with multiple containers, specify the container name with `-c`:

```
kubectl exec nginx-lab-1-58f9bf94f7-jk85s -c <container-name> -- /bin/bash
```

Day 2: Deployment Strategies

Labels and Selectors

Labels are key-value pairs attached to Kubernetes objects (pods, deployments, services). Selectors are used to filter and target objects based on their labels.

Creating a Deployment and Working with Labels

```
# Create a deployment YAML file using dry-run
kubectl create deployment ipivrdeploy --image=bitnami/nginx:latest --dry-run=client -o
yaml > ipivrdeploy.yml

# Scale the deployment
kubectl scale deployment ipivrdeploy --replicas=5

# Exec into a pod
kubectl exec -it ipivrdeploy-74dddd4b6f-877gn -- /bin/bash
```

```
# View pods with node info
kubectl get pods -o wide

# View labels on deployments and pods
kubectl get deployments --show-labels
kubectl get pods --show-labels
```

Adding Labels

```
# Add a label to a deployment (only sets on deployment, NOT on replicaset or pod)

kubectl label deployment nginx-deploy tier=backend
```

Filtering with Selectors

```
# View all resources with their labels
oc get all --show-labels

# Filter resources by label
oc get all -l tier=backend
oc get all -l app=ipivrdeploy

# Delete all resources matching a label
oc delete all -l tier=backend
```

> **Note:** When you label a deployment, the label is only applied to the deployment object itself — it does not propagate down to the ReplicaSet or Pods. Pod labels are defined in the pod template (`spec.template.metadata.labels`).

Theory: How to Deploy and Update Applications Safely

When you update an application in production, the key question is: **how do you replace the old version with the new one without affecting users?**

Kubernetes/OpenShift offers two primary strategies:

Recreate Strategy

```
Before:  [v1] [v1] [v1]    ← All pods running v1
Step 1:  [ ] [ ] [ ]        ← ALL old pods killed at once (DOWNTIME!)
Step 2:  [v2] [v2] [v2]    ← All new pods start
```

- **Downtime:** Yes, guaranteed
- **Use when:** Database migrations that can't run two versions, breaking schema changes, dev/test environments

RollingUpdate Strategy

```
Before:  [v1] [v1] [v1]          ← 3 pods serving traffic
Step 1:  [v1] [v1] [v1] [v2]    ← New pod starts (maxSurge: 1)
Step 2:  [v1] [v1] [v2]          ← Old pod removed after new is ready
Step 3:  [v1] [v1] [v2] [v2]    ← Next new pod starts
...continues until all replaced...
Final:   [v2] [v2] [v2]          ← Zero downtime achieved ✓
```

- **Downtime:** None (when configured correctly)
- **Use when:** Stateless services, APIs, web apps — most production workloads

---

Key Configuration Parameters Explained

Rolling Update Settings

```yaml
spec:
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

Parameter	Value	Meaning
`maxSurge`	`1`	Allow 1 EXTRA pod above replica count during update. Spins up new pod first, then

Parameter	Value	Meaning
		kills old one (“create before destroy”)
`maxUnavailable`	`0`	Never have fewer pods than replica count serving traffic. Guarantees zero downtime — old pod stays until new one passes readiness probe
`minReadySeconds`	`10`	Even after readiness probe passes, wait 10 more seconds before considering pod truly available. Safety buffer for apps that pass probe initially but crash seconds later

How They Work Together (3 replicas, minReadySeconds: 10)

```
Start:      [old] [old] [old]              → 3 running

Step 1:     [old] [old] [old] [NEW]        → 4 running (maxSurge allows +1)
            NEW pod starts, readiness probe passes at 5s...
            Waiting 10 more seconds (minReadySeconds)...
            At 15s: NEW is considered truly available ✓

Step 2:     [old] [old] [NEW]              → 3 running (1 old pod removed)

Step 3:     [old] [old] [NEW] [NEW2]       → 4 running (next new pod created)
            Readiness passes... wait 10s... NEW2 available ✓

Step 4:     [old] [NEW] [NEW2]             → 3 running (another old pod removed)

Step 5:     [old] [NEW] [NEW2] [NEW3]      → 4 running (last new pod created)
            Readiness passes... wait 10s... NEW3 available ✓

Step 6:     [NEW] [NEW2] [NEW3]            → 3 running □ Done!
```

Key takeaway: `maxSurge` and `maxUnavailable` control how many pods change at once. `minReadySeconds` controls how long to wait between each change.

Graceful Shutdown: terminationGracePeriodSeconds & preStop

```yaml
spec:
  terminationGracePeriodSeconds: 30
  containers:
  - name: nginx
    lifecycle:
      preStop:
        exec:
          command: ["sleep", "5"]
```

Parameter	Value	Meaning
`terminationGracePeriodSeconds`	`30`	Total time allowed for pod to shut down gracefully. After 30s, Kubernetes sends SIGKILL (hard kill)
`preStop`	`sleep 5`	Command that runs BEFORE SIGTERM is sent. Delays shutdown so load balancer can remove the pod first

Why preStop: sleep 5?

When a pod is terminated, two things happen **simultaneously**: 1. Kubernetes sends SIGTERM to the container 2. Kubernetes tells the load balancer to stop sending traffic

The problem: load balancer update takes a few seconds to propagate. During that gap, traffic still routes to a dying pod → **failed requests**.

`preStop: sleep 5` delays the shutdown, giving the load balancer time to drain.

Termination Timeline

```
0s   — Kubernetes decides to terminate the pod
0s   — preStop runs (sleep 5) AND load balancer starts removing the pod
5s   — preStop finishes, SIGTERM sent to the app
5s   — App starts graceful shutdown (finishes in-flight requests)
5-30s — App shuts down gracefully
30s  — If still running, SIGKILL (force kill)
       ↑ terminationGracePeriodSeconds limit
```

Important rule:

```
terminationGracePeriodSeconds ≥ preStop sleep + app shutdown time
         30s                  ≥     5s        +      10s          ✓
```

Hands-On: Test Rolling Update (Zero Downtime)

rolling-update-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-rolling
  labels:
    app: nginx-rolling
spec:
  replicas: 3
  minReadySeconds: 0
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: nginx-rolling
  template:
    metadata:
      labels:
        app: nginx-rolling
    spec:
      terminationGracePeriodSeconds: 30
      containers:
      - name: nginx
        image: docker.io/nginxinc/nginx-unprivileged:1.25
        ports:
        - containerPort: 8080
        readinessProbe:
          httpGet:
            path: /
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 3
        livenessProbe:
          httpGet:
            path: /
            port: 8080
          initialDelaySeconds: 10
```

```
            periodSeconds: 5
          lifecycle:
            preStop:
              exec:
                command: ["sleep", "5"]
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-rolling-svc
spec:
  selector:
    app: nginx-rolling
  ports:
  - port: 8080
    targetPort: 8080
---
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: nginx-rolling
spec:
  to:
    kind: Service
    name: nginx-rolling-svc
  port:
    targetPort: 8080
```

Step-by-Step

Deploy:

```
oc apply -f rolling-update-deployment.yaml
oc rollout status deployment/nginx-rolling
oc get pods -l app=nginx-rolling
```

Start downtime monitor (Terminal 1):

```
ROLLING_URL=$(oc get route nginx-rolling -o jsonpath='{.spec.host}')
while true; do
  CODE=$(curl -o /dev/null -s -w '%{http_code}' --max-time 2 http://$ROLLING_URL)

  if [ "$CODE" = "200" ]; then
    echo "$(date '+%Y-%m-%d %H:%M:%S') SUCCESS: $CODE"
  else
    echo "$(date '+%Y-%m-%d %H:%M:%S') FAIL:    $CODE"
  fi
  sleep 0.1
done
```

Trigger rolling update (Terminal 2):

```
        oc describe deployment nginx-rolling | grep -i image
        oc set image deployment/nginx-rolling nginx=docker.io/nginxinc/nginx-unprivileged:1.27

        oc rollout status deployment/nginx-rolling
        oc get pods -l app=nginx-rolling -w
        oc describe deployment nginx-rolling | grep -i image
        kubectl rollout history deployment/nginx-rolling
```

Expected result: Zero or near-zero errors in Terminal 1.

Rollback if needed:

```
        oc rollout undo deployment/nginx-rolling
        kubectl rollout history deployment/nginx-rolling
        oc describe deployment nginx-rolling | grep -i image
```

Deployment History and Rollback

To record the cause of a revision, annotate the deployment with `kubernetes.io/change-cause`:

```
        kubectl annotate deployment/nginx-rolling kubernetes.io/change-cause="initial
deployment with image 1.25"
        kubectl rollout history deployment/nginx-rolling
```

Cleanup:

```
        oc delete -f rolling-update-deployment.yaml
```

Hands-On: Test Recreate Strategy (Expected Downtime)

recreate-deployment.yaml

```
        apiVersion: apps/v1
        kind: Deployment
        metadata:
          name: nginx-recreate
          labels:
```

```
      app: nginx-recreate
  spec:
    replicas: 3
    strategy:
      type: Recreate
    selector:
      matchLabels:
        app: nginx-recreate
    template:
      metadata:
        labels:
          app: nginx-recreate
      spec:
        containers:
        - name: nginx
          image: docker.io/nginxinc/nginx-unprivileged:1.25
          ports:
          - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 3
  ---
  apiVersion: v1
  kind: Service
  metadata:
    name: nginx-recreate-svc
  spec:
    selector:
      app: nginx-recreate
    ports:
    - port: 8080
      targetPort: 8080
  ---
  apiVersion: route.openshift.io/v1
  kind: Route
  metadata:
    name: nginx-recreate
  spec:
    to:
      kind: Service
      name: nginx-recreate-svc
    port:
      targetPort: 8080
```

Step-by-Step

Deploy:

```
oc apply -f recreate-deployment.yaml
oc rollout status deployment/nginx-recreate
```

Start downtime monitor (Terminal 1):

```
RECREATE_URL=$(oc get route nginx-recreate -o jsonpath='{.spec.host}')
while true; do
  CODE=$(curl -o /dev/null -s -w '%{http_code}' --max-time 2 http://$RECREATE_URL)

  if [ "$CODE" = "200" ]; then
    echo "$(date '+%Y-%m-%d %H:%M:%S') SUCCESS: $CODE"
  else
    echo "$(date '+%Y-%m-%d %H:%M:%S') FAIL:    $CODE"
  fi
  sleep 0.1
done
```

Trigger recreate deployment (Terminal 2):

```
oc set image deployment/nginx-recreate nginx=docker.io/nginxinc/nginx-unprivileged:1.27

oc get pods -l app=nginx-recreate -w
```

Expected result: Stream of FAIL messages (503/000) — ALL pods terminate before new ones start.

Cleanup:

```
oc delete -f recreate-deployment.yaml
```

Day 3: Configuration, Secrets & Health Probes

Theory: Externalizing Configuration

Applications need configuration (database URLs, feature flags, API keys). Hardcoding them in container images is bad practice because: - You'd need a different image per environment - Secrets would be baked into the image layer - Changing config requires rebuilding

Kubernetes solves this with **ConfigMaps** and **Secrets**.

ConfigMap vs Secret

	ConfigMap	Secret
Stored as	Plain text	Base64 encoded
Visible in `oc describe`?	Yes	No (hidden)
Use for	Non-sensitive config	Passwords, API keys, certificates
Size limit	1 MiB	1 MiB

Two Ways to Consume Them

Method	Auto-updates?	Use case
Environment variable	No (needs pod restart)	Simple key-value config
Volume mount	Yes (~30s delay)	Config files (nginx.conf, etc.)

Hands-On: ConfigMaps & Secrets

Step 1: Create ConfigMap and Secret

configmap-secret.yaml:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  APP_DEBUG: "false"
  nginx.conf: |
    server {
      listen 8080;
```

```
      location / {
        return 200 'ConfigMap works!\n';
        add_header Content-Type text/plain;
      }
    }
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:
  DB_PASSWORD: "myS3cretP@ss"
  API_KEY: "abc123-secret-key"
```

```
oc apply -f configmap-secret.yaml
oc get configmap app-config
oc get secret app-secret
oc describe configmap app-config
oc get secret app-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
```

Step 2: Deploy Application That Consumes Them

configmap-secret-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: configmap-secret-test
  labels:
    app: configmap-secret-test
spec:
  replicas: 1
  selector:
    matchLabels:
      app: configmap-secret-test
  template:
    metadata:
      labels:
        app: configmap-secret-test
    spec:
      containers:
      - name: nginx
        image: docker.io/nginxinc/nginx-unprivileged:1.25
        ports:
        - containerPort: 8080
        env:
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: app-config
```

```
                key: APP_ENV
          - name: APP_DEBUG
            valueFrom:
              configMapKeyRef:
                name: app-config
                key: APP_DEBUG
          - name: DB_PASSWORD
            valueFrom:
              secretKeyRef:
                name: app-secret
                key: DB_PASSWORD
          - name: API_KEY
            valueFrom:
              secretKeyRef:
                name: app-secret
                key: API_KEY
          volumeMounts:
          - name: config-volume
            mountPath: /etc/nginx/conf.d
          - name: secret-volume
            mountPath: /etc/secrets
            readOnly: true
        volumes:
        - name: config-volume
          configMap:
            name: app-config
            items:
            - key: nginx.conf
              path: default.conf
        - name: secret-volume
          secret:
            secretName: app-secret
```

```
oc apply -f configmap-secret-deployment.yaml
oc get pods -l app=configmap-secret-test -w
```

Step 3: Verify Environment Variables

```
POD=$(oc get pods -l app=configmap-secret-test -o
jsonpath='{.items[0].metadata.name}')

# ConfigMap env vars
oc exec $POD -- env | grep APP_
# Expected: APP_ENV=production, APP_DEBUG=false

# Secret env vars
oc exec $POD -- env | grep -E "DB_PASSWORD|API_KEY"
# Expected: DB_PASSWORD=myS3cretP@ss, API_KEY=abc123-secret-key
```

Step 4: Verify Volume Mounts

```
# ConfigMap as file
oc exec $POD -- cat /etc/nginx/conf.d/default.conf

# Secret as files
oc exec $POD -- ls /etc/secrets/
oc exec $POD -- cat /etc/secrets/DB_PASSWORD
```

Step 5: Test Live ConfigMap Update

```
# Update ConfigMap
oc patch configmap app-config -p '{"data":{"APP_ENV":"staging"}}'

# Volume mounts auto-update (~30 seconds)
# But env vars do NOT auto-update — requires pod restart:
oc rollout restart deployment/configmap-secret-test
POD=$(oc get pods -l app=configmap-secret-test -o jsonpath='{.items[0].metadata.name}')

oc exec $POD -- env | grep APP_ENV
# Expected: APP_ENV=staging
```

Step 6: Test Missing Secret (Error Handling)

```
oc delete secret app-secret
oc rollout restart deployment/configmap-secret-test
oc get pods -l app=configmap-secret-test
# Expected: Pod stuck in CreateContainerConfigError

# Fix it:
oc apply -f configmap-secret.yaml
```

Cleanup

```
oc delete -f configmap-secret-deployment.yaml
oc delete -f configmap-secret.yaml
```

Health Probes: Keeping Applications Alive

What Are Probes?

Kubernetes uses probes to monitor container health:

Probe	Question it answers	On failure
Readiness	“Can this pod serve traffic?”	Pod removed from Service (no traffic sent)
Liveness	“Is this pod still alive?”	Pod is restarted
Startup	“Has this pod started yet?”	Other probes disabled until it passes

How They Work Together

```
Pod starts → Startup probe checks → [passes] → Readiness + Liveness begin
                                    [fails]  → Container restarted

Running:
  Readiness fails → Pod removed from Service endpoints (stays alive)
  Liveness fails  → Container killed and restarted
```

Key Difference

- **Readiness failure** = “Don’t send me traffic” (pod stays alive, just excluded)
- **Liveness failure** = “I’m dead, restart me” (pod is killed and recreated)

Probe Settings Explained

Setting	Meaning
`httpGet.path`	The HTTP endpoint to check (e.g., `/`)
`httpGet.port`	The port to send the health check request to
`initialDelaySeconds`	How long to wait after container starts before first probe check
`periodSeconds`	How often (in seconds) to perform the probe

Example breakdown:

```yaml
        readinessProbe:
          httpGet:
            path: /
            port: 8080
          initialDelaySeconds: 5    # Wait 5s after start, then begin checking
          periodSeconds: 3           # Check every 3s — fail detected quickly

        livenessProbe:
          httpGet:
            path: /
            port: 8080
          initialDelaySeconds: 10   # Wait 10s (give app time to start before killing it)

          periodSeconds: 5           # Check every 5s — less aggressive than readiness
```

> **Why different timings?** Readiness checks start sooner and more frequently because removing a pod from traffic is low-cost. Liveness starts later and checks less often because a failure triggers a container restart — you want to avoid false positives.

Hands-On: Health Probe Testing

Prerequisites

Ensure your rolling deployment is running with probes configured:

```
        oc apply -f rolling-update-deployment.yaml
        oc get pods -l app=nginx-rolling
```

Start Traffic Monitor (Terminal 1)

```
        ROLLING_URL=$(oc get route nginx-rolling -o jsonpath='{.spec.host}')
        while true; do
          CODE=$(curl -o /dev/null -s -w '%{http_code}' --max-time 2 http://$ROLLING_URL)

          if [ "$CODE" = "200" ]; then
            echo "$(date '+%Y-%m-%d %H:%M:%S') SUCCESS: $CODE"
          else
            echo "$(date '+%Y-%m-%d %H:%M:%S') FAIL:    $CODE"
          fi
          sleep 0.1
        done
```

Test 1: Readiness Probe Failure Leading to Restart

What should happen: Pod becomes NotReady → removed from Service → traffic goes to other pods → liveness probe also fails → container is restarted → pod recovers

```
        POD=$(oc get pods -l app=nginx-rolling -o
jsonpath='{.items[0].metadata.name}')

        # Break readiness by removing nginx config
        oc exec $POD -- mv /etc/nginx/conf.d/default.conf /etc/nginx/conf.d/default.conf.bak

        oc exec $POD -- nginx -s reload
```

Observe:

```
        oc get pods -l app=nginx-rolling -w
```

Expected: - Pod shows `0/1 READY` (readiness fails first → removed from traffic) - Traffic monitor shows **zero failures** (other 2 pods continue serving) - Liveness probe then fails → container is **restarted** (RESTARTS increases by 1) - After restart, default config is restored, pod becomes `1/1 READY` again

> **Why does it restart?** Both readiness and liveness use the same `httpGet` on port 8080. When nginx stops serving, both probes fail. Readiness removes the pod from traffic quickly, and liveness restarts it shortly after. To test readiness-only failure, you would need separate probe endpoints.

Test 2: Liveness Probe Failure (Container RESTARTED)

What should happen: Liveness fails → Kubernetes restarts the container

```
        POD=$(oc get pods -l app=nginx-rolling -o
jsonpath='{.items[0].metadata.name}')

        # Kill nginx process — port 8080 unreachable
        oc exec $POD -- nginx -s stop
```

Observe:

```
        oc get pods -l app=nginx-rolling -w
```

Expected: - Readiness fails first → pod removed from traffic - Liveness fails after ~15 seconds → container **restarted** - RESTARTS column increases by 1 - After restart, pod rejoins Service - Traffic monitor shows **zero or near-zero failures**

Test 3: All Probes Fail on ALL Pods (Downtime!)

```
        for POD in $(oc get pods -l app=nginx-rolling -o
jsonpath='{.items[*].metadata.name}'); do
          oc exec $POD -- nginx -s stop
        done
```

Expected: Traffic monitor shows FAIL messages until all pods restart and pass readiness probe.

> **Note:** You may not see any failures in practice. The `for` loop stops pods sequentially — by the time the last pod is stopped, the first may have already been restarted by the liveness probe. Fast-starting containers like nginx recover in seconds, making it difficult to observe actual downtime. This demonstrates that lightweight containers with proper health probes are resilient even in worst-case scenarios.

Probe Testing Summary

Test	What you do	What happens	Restarted?	Traffic impact
Remove config + reload	nginx stops serving on `/`	Pod removed from Service, then liveness fails	**Yes**	Zero (other pods serve)
`nginx -s stop`	Port 8080 unreachable	Container restarted	**Yes**	Zero (other pods serve)
Stop ALL pods	All ports unreachable	All containers restarted	**Yes**	Minimal or none — fast-restarting containers recover before all pods fail

Key Takeaway

With 3 replicas, losing 1 pod causes **zero user impact** — this is why we run multiple replicas! Even stopping all pods may not cause downtime if the container restarts faster than the sequential failure propagates.

Cleanup

```
oc delete -f rolling-update-deployment.yaml
```

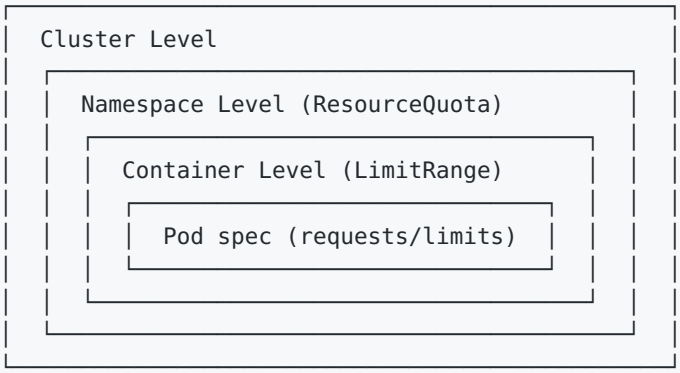
Day 4: Resource Management

Theory: Governing Cluster Resources

In a shared cluster, without resource controls: - One team's runaway app can starve others - Pods might be scheduled on nodes without enough capacity - OOM (Out of Memory) kills happen unpredictably

Kubernetes solves this with a layered approach:

Resource Hierarchy



Core Concepts

Resource	Purpose
requests	Minimum guaranteed resources; used by scheduler for pod placement
limits	Maximum resources a container can use; enforced at runtime
ResourceQuota	

Resource	Purpose
	Namespace-level cap on total resource consumption
LimitRange	Per-container defaults, min, and max constraints

Requests vs Limits — What Happens?

	CPU	Memory
Below request	Gets guaranteed CPU time	Gets guaranteed memory
Between request and limit	Gets extra CPU if available (burstable)	Uses extra memory if available
At limit	Throttled (slowed down)	OOMKilled (container restarted)
No request/ limit set	BestEffort — first to be evicted	BestEffort — first to be OOMKilled

Key insight: - CPU is **compressible** — exceeding limit just throttles, doesn't kill - Memory is **incompressible** — exceeding limit triggers OOMKill

---

Hands-On: Resource Quotas & Limits

> **Role Legend:** - [ADMIN] **Admin** — Requires `admin` or `cluster-admin` role (NOT available on OpenShift Sandbox) - [DEV] **Developer** — Works with `edit` / `developer` role (available on OpenShift Sandbox)

Prerequisites

```
# [ADMIN] Admin: Create a new project (on self-managed clusters)
oc new-project resource-demo
```

```
# [DEV] Developer (Sandbox alternative): Use your assigned namespace
oc project <your-username>-dev
```

Step 1: Apply LimitRange

> [ADMIN] **Admin only** — On OpenShift Sandbox, LimitRanges are pre-configured by the platform.

limit-range.yaml:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
spec:
  limits:
    - type: Container
      default:
        cpu: 500m
        memory: 256Mi
      defaultRequest:
        cpu: 100m
        memory: 128Mi
      max:
        cpu: "1"
        memory: 1Gi
      min:
        cpu: 50m
        memory: 64Mi
```

```
# [ADMIN] Admin: Create the LimitRange
oc apply -f limit-range.yaml

# [DEV] Developer: View the existing LimitRange (works on Sandbox)
oc describe limitrange
```

This sets: - **Default limits** (500m CPU, 256Mi memory) for containers that don't specify their own - **Default requests** (100m CPU, 128Mi memory) - **Max allowed** (1 CPU, 1Gi memory) — reject anything above - **Min allowed** (50m CPU, 64Mi memory) — reject anything below

---

Step 2: Apply ResourceQuota

> [ADMIN] **Admin only** — On OpenShift Sandbox, ResourceQuotas are pre-configured by the platform.

resource-quota.yaml:

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.cpu: "4"
    limits.memory: 4Gi
    pods: "10"
```

```
# [ADMIN] Admin: Create the ResourceQuota
oc apply -f resource-quota.yaml

# [DEV] Developer: View existing quotas (works on Sandbox)
oc describe quota
```

This caps the entire namespace: - Total CPU requests: 2 cores - Total memory requests: 2Gi - Total CPU limits: 4 cores - Total memory limits: 4Gi - Maximum 10 pods

---

Step 3: Deploy with Explicit Resources

> [DEV] **Developer** — Works on OpenShift Sandbox.

resource-limits-deployment.yaml:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: resource-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: resource-demo
  template:
    metadata:
```

```
      labels:
        app: resource-demo
    spec:
      containers:
        - name: app
          image: docker.io/nginxinc/nginx-unprivileged:1.25
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 250m
              memory: 256Mi
          ports:
            - containerPort: 8080
```

> **Note:** Use `nginxinc/nginx-unprivileged` instead of `nginx` — OpenShift blocks containers that run as root.

```
# [DEV] Developer
oc apply -f resource-limits-deployment.yaml
oc get pods
oc describe quota
```

Notice quota usage now reflects 2 replicas × their requests/limits.

Step 4: Test Quota Enforcement

> [DEV] **Developer** — Works on OpenShift Sandbox.

On Sandbox, the pre-configured quota has `requests.cpu: 3` as the hard limit. To trigger exhaustion, deploy pods with large CPU requests that exceed the remaining capacity:

```
# [DEV] Developer: Create a deployment designed to exhaust CPU quota
oc create deployment quota-buster --image=docker.io/nginxinc/nginx-unprivileged:1.25 --replicas=0
oc set resources deployment quota-buster --requests=cpu=500m,memory=128Mi --limits=cpu=1,memory=256Mi
oc scale deployment quota-buster --replicas=10

# Check events for failures
oc get events --field-selector reason=FailedCreate
```

Expected: Events show `exceeded quota: compute-deploy, requested: requests.cpu=500m, used: requests.cpu=3, limited: requests.cpu=3` — new pods cannot be created.

> **Why replicas=0 first?** Setting resources before scaling ensures all pods attempt to schedule with the resource requests applied. If you scale first and set resources after, rolling updates free old pods one-by-one, masking the quota failure.

Step 5: Test LimitRange Defaults

> [DEV] **Developer** — Works on OpenShift Sandbox.

```
        # [DEV] Developer: Deploy a pod WITHOUT specifying resources
        oc run bare-pod --image=docker.io/nginxinc/nginx-unprivileged:1.25 --restart=Neve
r

        # Check what resources were injected by the LimitRange
        oc describe pod bare-pod | grep -A4 "Limits\|Requests"
```

Expected: LimitRange automatically injects default requests and limits defined by the cluster admin.

Step 6: Test LimitRange Min/Max Enforcement

> [DEV] **Developer** — Works on OpenShift Sandbox.

```
        # [DEV] Developer: Try creating a pod that exceeds the LimitRange max
        oc run over-limit --image=docker.io/nginxinc/nginx-unprivileged:1.25 --restart=Never \

          --overrides='{"spec":{"containers":[{"name":"over-limit","image":"docker.io/nginxinc/
nginx-unprivileged:1.25","resources":{"requests":{"cpu":"2","memory":"2Gi"}}}]}}'
```

Expected: Rejected with a Forbidden error referencing the LimitRange (or quota exceeded if the request surpasses the namespace quota).

Step 7: Monitor Resource Usage

> [DEV] **Developer** — `oc adm top pods` works on Sandbox. `oc adm top nodes` requires cluster-admin.

```
# [DEV] Developer: Pod-level usage
oc adm top pods

# [ADMIN] Admin only: Node-level usage
oc adm top nodes

# [DEV] Developer: Quota status
oc describe quota
```

Cleanup

```
# [ADMIN] Admin (self-managed cluster):
oc delete project resource-demo

# [DEV] Developer (Sandbox): Delete individual resources
oc delete deployment resource-demo quota-buster
oc delete pod bare-pod over-limit --ignore-not-found
```

Key Takeaways

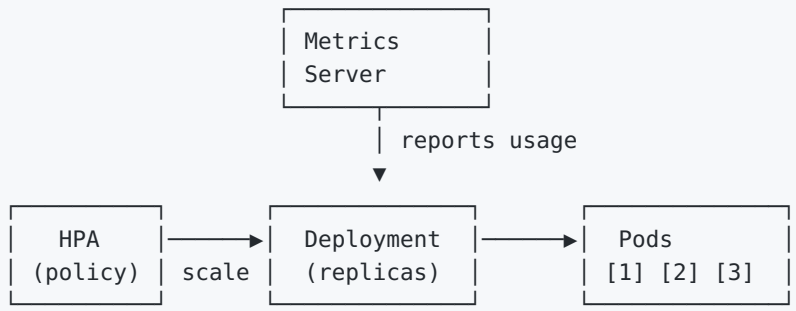
- **Requests** affect scheduling; **limits** affect runtime enforcement
- A **ResourceQuota** prevents a namespace from consuming more than its fair share
- A **LimitRange** ensures every container has sensible defaults and prevents outliers
- Always set requests — HPA uses them as baseline to calculate utilization percentage
- CPU over-limit = throttled; Memory over-limit = OOMKilled
- On **OpenShift Sandbox**, LimitRanges and ResourceQuotas are pre-configured — use `oc describe limitrange` and `oc describe quota` to inspect them
- Use `nginxinc/nginx-unprivileged` image on OpenShift (standard `nginx` requires root and will fail)

Day 5: Autoscaling (HPA) & Wrap-up

Theory: Horizontal Pod Autoscaler

The Horizontal Pod Autoscaler (HPA) automatically scales the number of pods based on observed CPU/memory utilization or custom metrics.

How HPA Works




```
HPA loop (every 15s):
1. Get current metrics from Metrics Server
2. Calculate: current utilization / target utilization
3. Desired replicas = current replicas × (current / target)
4. Scale up or down (respecting min/max)
```

Scaling Formula

```
desiredReplicas = ceil(currentReplicas × (currentMetricValue / desiredMetricValue))
```

Example: 2 pods at 80% CPU, target is 50%:

```
desiredReplicas = ceil(2 × (80/50)) = ceil(3.2) = 4 pods
```

Requirements for HPA

1. **resources.requests must be defined** — HPA calculates % based on requests
2. **Metrics Server must be running** — Available by default in OpenShift

---

Hands-On: HPA Testing

Step 1: Deploy the Application

hpa-deployment.yaml:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hpa-test-app
  labels:
    app: hpa-test-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: hpa-test-app
  template:
    metadata:
      labels:
        app: hpa-test-app
    spec:
      containers:
      - name: app
        image: docker.io/nginxinc/nginx-unprivileged:1.25
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: 50m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 256Mi
```

```
oc apply -f hpa-deployment.yaml
oc get pods -l app=hpa-test-app -w
```

Step 2: Create the HPA

hpa.yaml:

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hpa-test-app
spec:
```

```
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hpa-test-app
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 70
```

```
oc apply -f hpa.yaml
oc get hpa
oc describe hpa hpa-test-app
```

Step 3: Check Current Usage

```
oc adm top pods -l app=hpa-test-app
oc get hpa hpa-test-app -w
```

Step 4: Generate CPU Load to Trigger Scaling

Option A — Exec into the pod:

```
POD=$(oc get pods -l app=hpa-test-app -o jsonpath='{.items[0].metadata.name}')
oc exec $POD -- /bin/sh -c "while true; do true; done" &
```

Option B — Create a load generator pod:

```
oc expose deployment hpa-test-app --port=8080
oc run load-generator --image=busybox --restart=Never -- /bin/sh -c "while true; do
wget -q -O- http://hpa-test-app:8080; done"
```

Step 5: Watch HPA Scale Up

```
# Watch HPA react (another terminal)
oc get hpa hpa-test-app -w

# Watch pods scale up
oc get pods -l app=hpa-test-app -w

# Monitor CPU/memory continuously
watch oc adm top pods -l app=hpa-test-app
```

Step 6: Check HPA Events

```
oc describe hpa hpa-test-app | grep -A 20 "Events"
```

Step 7: Stop Load and Watch Scale Down

```
# Kill the load generator
oc delete pod load-generator --ignore-not-found

# Or restart if you used exec method
oc rollout restart deployment/hpa-test-app

# Watch scale down (~5 minutes cooldown)
oc get hpa hpa-test-app -w
oc get pods -l app=hpa-test-app -w
```

HPA Configuration Summary

Setting	Value	Meaning
minReplicas	1	Minimum pods running
maxReplicas	5	Maximum pods HPA can scale to
CPU target	50%	Scale up when avg CPU > 50% of request (25m)
Memory target	70%	Scale up when avg memory > 70% of request (~45Mi)
Scale down delay	~5 min	Default cooldown before scaling down

Cleanup

```
oc delete -f hpa.yaml
oc delete -f hpa-deployment.yaml
oc delete svc hpa-test-app --ignore-not-found
oc delete pod load-generator --ignore-not-found
```

5-Day Training Recap

Day	Topic	Key Skill
1	Foundations	Understand K8s/OpenShift architecture, deploy first app
2	Deployment Strategies	Zero-downtime rolling updates, rollbacks
3	Config & Probes	Externalize config, self-healing with probes
4	Resource Management	Quotas, limits, cluster governance
5	Autoscaling	HPA, metrics-based scaling

What’s Next?

- **Operators** — Manage complex stateful applications
- **GitOps (ArgoCD)** — Declarative, git-driven deployments
- **Tekton Pipelines** — Cloud-native CI/CD
- **Service Mesh (Istio)** — Advanced traffic management, observability
- **Network Policies** — Fine-grained pod-to-pod network rules

---

End of Training Material